MODULE 6 – BUILD TOOLS, IDE AND DEBUGGING

1. Introduction to Build Tools

Build tools are software programs used to compile source code, manage dependencies, execute tests and prepare applications for deployment.

In .NET development, build tools help convert C# source code into executable application output.

Why do we need Build Tools?

- Compile source code.
- Detect compilation errors.
- Restore project dependencies.
- Build applications automatically.
- Execute automated tests.
- Prepare applications for deployment.
- Manage different build configurations.

Examples of .NET Build Tools:

1. .NET CLI
2. MSBuild
3. C# Compiler (Roslyn)
4. NuGet
5. Visual Studio Build Tools

2. What is an IDE?

IDE stands for Integrated Development Environment.

An IDE is a software application that provides all the essential tools required to develop, build, run, test and debug applications in one environment.

Features of an IDE:

- Code editor.
- Syntax highlighting.
- IntelliSense.
- Code completion.

- Project management.
- Build and run support.
- Debugging tools.
- Error detection.
- Integrated terminal.
- Extensions and plugins.

Examples:

- Visual Studio
- Visual Studio Code
- JetBrains Rider
- Eclipse
- IntelliJ IDEA

3. Visual Studio

Visual Studio is an Integrated Development Environment developed by Microsoft.

It is widely used for developing C#, .NET, ASP.NET Core, Web API, MVC and desktop applications.

Important Features:

1. IntelliSense: Provides code suggestions and automatic completion.
2. Syntax Highlighting: Displays different code elements using different colors.
3. Solution Explorer: Displays the project and file structure.
4. Error List: Displays errors and warnings.
5. Debugger: Helps identify and fix errors.
6. NuGet Package Manager: Installs and manages packages.
7. Git Integration: Provides source control features.
8. Integrated Terminal: Allows executing command-line instructions.
9. Code Navigation: Helps navigate between classes, methods and definitions.
10. Build Tools: Helps compile and build applications.

4. Visual Studio vs Visual Studio Code

Visual Studio:

- Full-featured IDE.
- Provides integrated project templates.
- Has a powerful graphical debugger.
- Includes built-in build and testing support.
- Commonly used for enterprise .NET development.

- Available primarily for Windows, with a separate Visual Studio for Mac product that was retired in 2024.

Visual Studio Code:

- Lightweight, extensible code editor.
- Supports multiple programming languages.
- Requires extensions for additional development features.
- Provides debugging through language extensions.
- Supports an integrated terminal.
- Available on Windows, Linux and macOS.

Difference:

Visual Studio is a full IDE with extensive integrated development features, whereas Visual Studio Code is a lightweight editor that can be extended using extensions.

5. Visual Studio Important Windows

Visual Studio provides several windows that help developers during development.

1. Solution Explorer

Displays:

- Solution files.
- Projects.
- Controllers.
- Models.
- Views.
- Services.
- Configuration files.
- Other project files.

2. Error List

Displays:

- Compilation errors.
- Warnings.
- Informational messages.

3. Output Window

Displays build, debugging and application output messages.

4. Package Manager Console

Used to execute NuGet and Entity Framework Core commands.

Example:

```
None
Add-Migration InitialCreate
Update-Database
```

5. Terminal

Used to execute commands such as:

```
None
dotnet build
dotnet run
git status
```

6. Properties Window

Displays and allows editing properties of the selected project or item.

6. Understanding a .NET Solution and Project

Solution:

A solution is a container that organizes one or more related projects.

Example:

```
None
StudentManagement.sln
|
├── StudentManagement
|   ├── Controllers
|   ├── Models
|   ├── Services
|   ├── Program.cs
|   └── StudentManagement.csproj
```

```
│
└── StudentManagement.Tests
    ├── CalculatorTests.cs
    └── StudentManagement.Tests.csproj
```

Project:

A project contains source code, configuration and dependency information required to build a particular application or library.

Important project file:

```
None
StudentManagement.csproj
```

Difference:

- Solution: Organizes multiple projects.
- Project: Contains the code and configuration for a particular application or library.

7. .NET SDK

SDK stands for Software Development Kit.

The .NET SDK provides the tools required to create, build, test and publish .NET applications.

It includes:

- .NET CLI.
- C# compiler.
- MSBuild.
- Project templates.
- Build-related tools.
- Required development components.

Check Installed SDK Version:

```
None
dotnet --version
```

Display SDK Information:

```
None
dotnet --info
```

List Installed SDKs:

```
None
dotnet --list-sdks
```

List Installed Runtimes:

```
None
dotnet --list-runtimes
```

8. .NET CLI

CLI stands for Command Line Interface.

The .NET CLI allows developers to create, build, run, test and publish .NET applications using terminal commands.

Important Commands:

Create a Console Application:

```
None
dotnet new console
```

Create an ASP.NET Core Web API:

```
None
dotnet new webapi
```

Create an MVC Application:

```
None
dotnet new mvc
```

Create a Class Library:

```
None
dotnet new classlib
```

Create an xUnit Test Project:

```
None
dotnet new xunit
```

Build a Project:

```
None
dotnet build
```

Run a Project:

```
None
dotnet run
```

Restore Dependencies:

```
None
dotnet restore
```

Run Tests:

```
None
dotnet test
```

Publish an Application:

```
None
dotnet publish
```

9. MSBuild

MSBuild stands for Microsoft Build Engine.

It is a build platform used by .NET projects to compile source code and produce application output.

It reads project and solution configuration files and executes the required build tasks.

Responsibilities of MSBuild:

- Compiles source code.
- Resolves project dependencies.
- Executes build tasks.
- Copies required files.
- Generates build output.
- Supports customized build processes.

Important MSBuild Files:

- .csproj
- .sln
- .props
- .targets

Example:

```
None
dotnet build
```

The .NET CLI uses MSBuild to perform the build process.

10. C# Compiler – Roslyn

Roslyn is the .NET compiler platform used for C# and Visual Basic.

It converts C# source code into Intermediate Language (IL).

Compilation Flow:

```
None
C# Source Code
      |
      v
C# Compiler (Roslyn)
      |
      v
Intermediate Language (IL)
      |
      v
.NET Runtime
      |
      v
JIT Compiler
      |
      v
Machine Code
```

Responsibilities:

- Checks code syntax.
- Performs semantic analysis.
- Identifies compilation errors.
- Produces Intermediate Language.
- Supports IDE features such as IntelliSense and code analysis.

11. NuGet Package Manager

NuGet is the package management system used by .NET.

It helps developers install and manage external libraries and dependencies.

Why do we use NuGet?

- Reuse existing libraries.
- Avoid writing everything from scratch.
- Manage package versions.
- Resolve dependencies.
- Simplify project development.

Examples of NuGet Packages:

- Microsoft.EntityFrameworkCore
- Microsoft.EntityFrameworkCore.SqlServer
- Microsoft.AspNetCore.Authentication.JwtBearer
- Microsoft.EntityFrameworkCore.Tools
- xUnit

Install a Package:

```
None
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
```

Remove a Package:

```
None
dotnet remove package Microsoft.EntityFrameworkCore.SqlServer
```

List Installed Packages:

```
None
dotnet list package
```

Restore Packages:

```
None
dotnet restore
```

12. Build Process in .NET

A build process converts source code and required dependencies into application output.

Main Steps:

1. Restore: Downloads required dependencies.
2. Compile: Converts source code into compiled output.
3. Build: Executes the required build steps.
4. Output: Generates compiled files in the output directory.

Command:

```
None
dotnet build
```

Important output folders:

```
None
bin/
obj/
```

bin Folder:

Contains compiled application output and related files.

obj Folder:

Contains intermediate build files, generated files and temporary build information.

These folders are usually excluded from source control using .gitignore.

13. Important .NET Build Commands

1. dotnet restore

Restores the dependencies required by the project.

```
None
dotnet restore
```

2. dotnet build

Builds the project and compiles its source code.

```
None
dotnet build
```

3. dotnet clean

Removes previous build output.

```
None
dotnet clean
```

4. dotnet rebuild

Cleans and then builds the project.

```
None
dotnet rebuild
```

5. dotnet run

Builds and runs the application.

```
None
dotnet run
```

6. dotnet publish

Prepares the application for deployment.

```
None
dotnet publish -c Release
```

14. Difference Between Build, Rebuild, Clean and Publish

Build:

- Compiles the project.
- Produces build output.
- Uses incremental build when possible.

Rebuild:

- Cleans existing build output.
- Builds the project again.

Clean:

- Removes previous build output.
- Does not compile the project.

Publish:

- Prepares the application for deployment.
- Produces the files needed for the selected deployment type.

15. Build Configurations

Build configurations determine how an application is compiled.

Two common configurations are:

1. Debug Configuration

Used during development.

Features:

- Supports debugging.
- Generates debugging information.
- Makes it easier to inspect variables and execution.
- Usually avoids some release optimizations.

2. Release Configuration

Used when preparing an application for production.

Features:

- Enables compiler optimizations where applicable.
- Produces optimized application output.
- Is intended for deployment and production scenarios.

Build in Debug Mode:

```
None
dotnet build -c Debug
```

Build in Release Mode:

```
None
dotnet build -c Release
```

Publish in Release Mode:

```
None
dotnet publish -c Release
```

16. Build Errors and Warnings

During compilation, the compiler may identify errors and warnings.

Build Error:

An error prevents the project from compiling successfully.

Example:

```
None
int number = "Hello";
```

This produces a type conversion error because a string cannot be assigned directly to an integer variable.

Warning:

A warning indicates a potential problem or an issue that should be reviewed.

Example:

```
None
int number = 10;
```

If the variable is never used, the compiler or analyzer may report a warning.

Difference:

- Error: Prevents successful compilation.
- Warning: Indicates a possible issue, but does not necessarily prevent compilation.

17. Introduction to Debugging

Debugging is the process of identifying, analyzing and fixing errors in a program.

A defect in a program is commonly called a bug.

Why do we Debug?

- Identify incorrect behavior.
- Find the source of errors.
- Understand the execution flow.
- Inspect variable values.
- Verify program logic.
- Improve application reliability.

Example:

```
None
int a = 10;
int b = 0;

int result = a / b;

Console.WriteLine(result);
```

This program causes a DivideByZeroException at runtime.

Debugging helps identify the exact line where the problem occurs.

18. Types of Errors

There are three commonly discussed types of programming errors.

1. Syntax Errors

Errors caused by incorrect programming language syntax.

Example:

```
None
Console.WriteLine("Hello"
```

The closing parenthesis is missing.

2. Runtime Errors

Errors that occur while the program is executing.

Example:

```
None
int a = 10;
int b = 0;

int result = a / b;
```

This causes a DivideByZeroException.

3. Logical Errors

Errors where the program executes without crashing but produces an incorrect result.

Example:

```
None
int length = 10;
int width = 5;

int area = length + width;
```

The correct formula for rectangle area is:

```
None
int area = length * width;
```

19. What is a Breakpoint?

A breakpoint is a debugging marker placed on a line of code.

When the program reaches a breakpoint during debugging, execution pauses at that point.

This allows developers to inspect variables and understand program execution.

How to Set a Breakpoint in Visual Studio?

1. Open the source code file.
2. Click the left margin beside the required line.
3. A red circle appears.
4. Start debugging.
5. Execution pauses when the breakpoint is reached.

Shortcut:

```
None
F9
```

F9 adds or removes a breakpoint on the current line.

20. Types of Breakpoints

1. Normal Breakpoint

Pauses execution whenever the selected line is reached.

2. Conditional Breakpoint

Pauses execution only when a specified condition is true.

Example:

```
None
if (age > 18)
{
    Console.WriteLine("Eligible");
}
```

A conditional breakpoint can be configured to stop only when age is greater than 18.

3. Hit Count Breakpoint

Pauses execution when a line is reached a specified number of times.

Useful when debugging loops.

4. Function Breakpoint

Pauses execution when a particular function or method is called.

21. Debugging Shortcuts in Visual Studio

Important keyboard shortcuts:

Shortcut	Purpose
F5	Start or continue debugging
Shift + F5	Stop debugging
Ctrl + F5	Run without debugging
F9	Add or remove breakpoint
F10	Step Over
F11	Step Into
Shift + F11	Step Out
Ctrl + Shift + F5	Restart debugging
Ctrl + Alt + Q	Open QuickWatch when supported

22. Step Over, Step Into and Step Out

These are important debugging operations.

1. Step Over (F10)

Executes the current line and moves to the next line without entering the called method.

Example:

```
None
int result = Add(10, 20);

Console.WriteLine(result);
```

When execution stops at the Add call, pressing F10 executes the method without stepping through its internal statements.

2. Step Into (F11)

Enters the called method to inspect its internal execution.

Example:

```
None
int result = Add(10, 20);
```

Pressing F11 takes you inside the Add method.

3. Step Out (Shift + F11)

Executes the remaining statements in the current method and returns to the calling method.

Remember:

- F10: Execute the line and move forward.
- F11: Enter the method.
- Shift + F11: Exit the current method.

23. Locals Window

The Locals window displays local variables available in the current execution scope.

It helps developers inspect variable values while debugging.

Example:

```
None
int age = 21;
string name = "Divya";
bool isStudent = true;
```

During debugging, the Locals window displays:

- age = 21
- name = "Divya"
- isStudent = true

24. Autos Window

The Autos window displays variables and expressions related to the current and previous statements in the debugging context.

It helps developers quickly inspect values relevant to the current execution point.

Difference Between Locals and Autos:

- Locals: Displays local variables in the current scope.
- Autos: Displays variables and expressions identified by the debugger as relevant to nearby statements.

25. Watch Window

The Watch window allows developers to monitor specific variables and expressions during debugging.

Example:

```
None
int a = 10;
int b = 20;

int result = a + b;
```

You can add the following expressions to the Watch window:

```
None
a
b
result
a + b
```

The debugger displays their values and updates them as execution progresses.

26. QuickWatch

QuickWatch is used to examine the value of a variable or expression during debugging.

Example:

```
None
int a = 10;
int b = 20;
```

You can select a variable or expression and open QuickWatch to inspect its current value.

27. Call Stack Window

The Call Stack window displays the sequence of method calls that led to the current execution point.

Example:

```
None
Main()
  |
  v
DisplayStudent()
  |
  v
CalculateAge()
```

If an error occurs inside CalculateAge(), the Call Stack helps identify how the program reached that method.

Why use Call Stack?

- Understand method execution flow.
- Identify the calling method.
- Track errors across multiple methods.
- Debug complex applications.

28. Immediate Window

The Immediate window allows developers to evaluate expressions and inspect or modify values during debugging.

Example:

```
None
int age = 21;
int mark = 90;
```

You can evaluate:

```
None
age
mark
age + mark
```

The window is especially useful for testing expressions without modifying the source code.

29. Output Window

The Output window displays messages generated during development and debugging.

It can show:

- Build messages.
- Debugging information.
- Diagnostic messages.
- Application output from debugging processes.

Example:

```
None
System.Diagnostics.Debug.WriteLine("Debug message");
```

This message can be viewed in the Visual Studio Output window when debugging.

30. Exception Handling During Debugging

An exception is an error that occurs while a program is executing.

Visual Studio allows developers to inspect exceptions when they occur.

Example:

```
None
try
{
    int a = 10;
    int b = 0;

    int result = a / b;
```

```
}
catch (DivideByZeroException ex)
{
    Console.WriteLine(ex.Message);
}
```

When debugging:

1. Set a breakpoint inside the try block.
2. Start the application.
3. Execute the statements one by one.
4. Observe the exception when division by zero occurs.
5. Inspect the exception details.

Exception Settings:

Visual Studio provides exception settings that control when the debugger breaks for particular exceptions.

Breaking when an exception is thrown can help identify the original source of the problem before it is handled.

31. Debugging an ASP.NET Core Application

ASP.NET Core applications can be debugged using Visual Studio.

Example:

Suppose you have a StudentController.

```
None
public class StudentController : Controller
{
    public IActionResult Details(int id)
    {
        string name = "Divya";

        return Content($"Student ID: {id}, Name: {name}");
    }
}
```

Debugging Steps:

1. Open StudentController.cs.
2. Set a breakpoint inside the Details method.
3. Start the application using F5.
4. Open the application in a browser.
5. Navigate to the Details route with a valid ID.
6. Execution pauses at the breakpoint.
7. Inspect the id and name values.
8. Use F10 and F11 to continue execution.

32. Launch Settings in ASP.NET Core

The launchSettings.json file contains development launch profiles for running and debugging an application.

Location:

```
None
Properties/launchSettings.json
```

Example:

```
None
{
  "profiles": {
    "https": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "applicationUrl":
"https://localhost:7057;http://localhost:5007",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}
```

Important properties:

- commandName: Specifies how the application is launched.
- launchBrowser: Determines whether to launch a browser.
- applicationUrl: Specifies development application URLs.
- environmentVariables: Defines environment variables for the launch profile.

Important:

launchSettings.json is mainly a local development configuration file. It is not automatically used as the production hosting configuration.

33. Development and Production Environments

ASP.NET Core applications commonly use different environments.

Development:

Used while developing an application.

- Helps developers debug applications.
- Provides detailed development error information.
- Supports development-specific configuration.

Staging:

Used to test the application in an environment similar to production before release.

Production:

Used when the application is deployed for actual users.

- Requires secure configuration.
- Should avoid exposing sensitive information.
- Uses production-ready settings.

Environment names are commonly configured using:

```
None
ASPNETCORE_ENVIRONMENT
```

34. Logging and Debugging

Logging records application events and execution information.

It helps developers understand application behavior without stopping execution at breakpoints.

Example:

```
None
public class StudentService
{
    private readonly ILogger<StudentService> _logger;

    public StudentService(ILogger<StudentService> logger)
    {
        _logger = logger;
    }

    public void GetStudent()
    {
        _logger.LogInformation("Getting student details");

        _logger.LogWarning("Student information is incomplete");

        _logger.LogError("Student retrieval failed");
    }
}
```

Important logging levels:

1. Trace
2. Debug
3. Information
4. Warning
5. Error
6. Critical

Difference Between Logging and Debugging:

Logging:

- Records application events.
- Helps monitor application behavior.
- Can be used in development and production.

Debugging:

- Allows developers to pause execution.

- Helps inspect variables and execution flow.
- Helps identify the source of errors.

35. Debugging with dotnet CLI

The .NET CLI provides useful commands for development and debugging.

Run the Application:

```
None
dotnet run
```

Watch for Code Changes:

```
None
dotnet watch
```

Build the Project:

```
None
dotnet build
```

Run Tests:

```
None
dotnet test
```

View Application Logs:

Application logging can be configured through logging providers and configuration settings. In a development environment, logs are commonly displayed in the terminal or IDE output window.

36. Hot Reload

Hot Reload allows developers to apply supported code changes while an application is running without performing a complete restart.

It improves development productivity.

Benefits:

- Reduces development time.
- Speeds up testing.
- Allows developers to see changes quickly.
- Reduces unnecessary application restarts.

In Visual Studio, Hot Reload is available for supported project types and code changes.

Using the CLI:

```
None
dotnet watch
```

The exact changes supported depend on the project type and runtime.

37. Debugging Database Operations with EF Core

Debugging is important when working with databases using Entity Framework Core.

Example:

```
None
var students = await _context.Students
    .Where(s => s.Age > 18)
    .ToListAsync();
```

When debugging this query, inspect:

- Whether DbContext is created correctly.
- Whether the database connection is valid.
- Whether the query condition is correct.
- Whether matching records exist.
- Whether any exception occurs.

Enable Sensitive Data Logging During Local Development:

```
None
builder.Services.AddDbContext<AppDbContext>(options =>
{
    options.UseSqlServer(

builder.Configuration.GetConnectionString("DefaultConnection"));
```

```
    options.EnableSensitiveDataLogging();
});
```

Sensitive data logging can expose confidential information in logs. Do not enable it in production without a specific security review.

38. Testing vs Debugging

Testing and debugging are related but different activities.

Testing:

Testing checks whether an application behaves as expected.

Example:

- Verify student registration.
- Verify login functionality.
- Verify calculation results.

Debugging:

Debugging identifies and fixes the reason for incorrect behavior.

Example:

- Student registration fails.
- A developer sets a breakpoint.
- Inspects submitted values.
- Identifies the incorrect validation condition.
- Fixes the issue.

Difference:

- Testing identifies failures or verifies behavior.
- Debugging investigates and fixes the underlying cause.

39. Common Development Problems and Solutions

1. Build Failed

Possible causes:

- Syntax errors.
- Missing dependencies.
- Incorrect project configuration.
- Type conversion errors.

Solution:

```
None
dotnet restore
dotnet build
```

Inspect the Error List for the exact issue.

2. Package Not Found

Possible causes:

- Package name is incorrect.
- Package source is unavailable.
- Dependencies have not been restored.

Solution:

```
None
dotnet restore
dotnet list package
```

3. Port Already in Use

Possible cause:

- Another application is already using the required port.

Solution:

- Stop the other application.
- Change the development application URL.
- Check which process is using the port.

4. Database Connection Failed

Possible causes:

- Incorrect connection string.

- SQL Server is not running.
- Incorrect database name.
- Authentication failure.

Solution:

- Verify the connection string.
- Check the SQL Server service.
- Test database connectivity.
- Check application logs.

5. Application Crashes at Runtime

Possible causes:

- Unhandled exception.
- Null reference.
- Invalid input.
- Database operation failure.

Solution:

- Set breakpoints.
- Inspect variable values.
- Check the Call Stack.
- Review exception details.
- Correct the underlying issue.